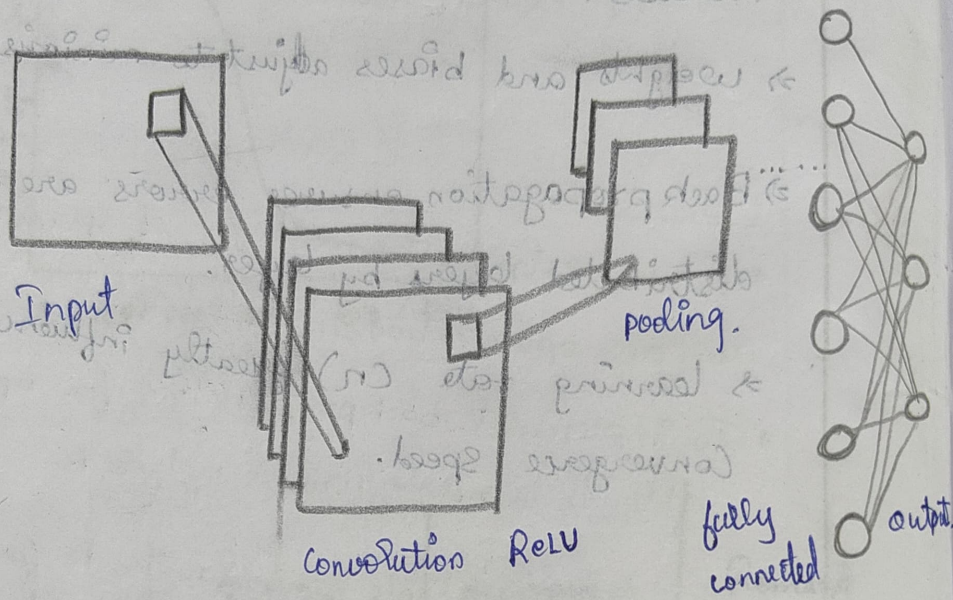
INDEX

NAME: Niranjan Gobe. S ROLL NO.: 008

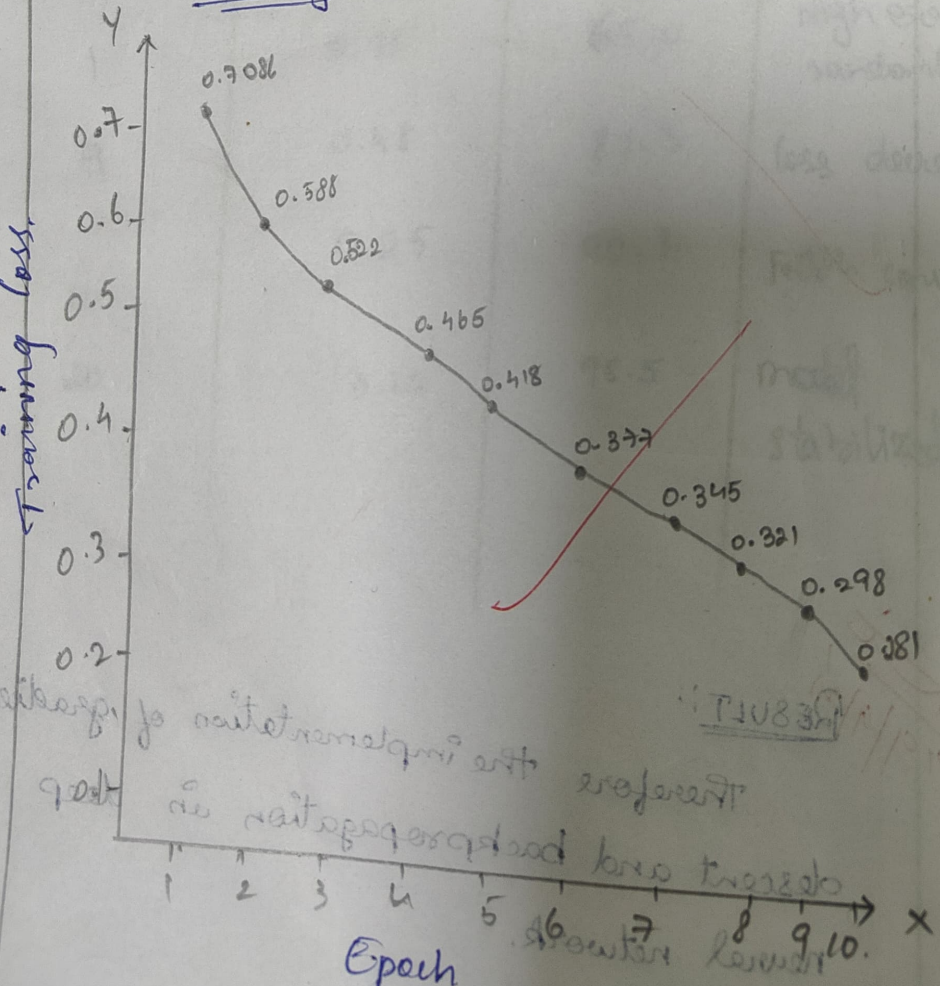
STD.: DIV./SEC.: SUBJECT:

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
1.	24.07.25	Exploring The Deep Learning Platforms.		
2.	7.08.25	Implement a classifier using open-source Dataset.		
3.	7.08.2025	Study of the classifier with respect to statistical parameters.		
4.	14.08.2025	Build a Single Feed Forward neural network to recognise handwritten characters.		
5.	22.08.25	Study of activation function and its role.		
6.	09.9.25	Implement gradient descent & backpropagation in deep neural network.		
7.	16.9.2025	Buid a CNN Model to classify Cat and Dog Image.		
8.	23.9.25	Experiment Using LSTM		
9.				

CNN Architecture Diagram:



Training Loss over Epoch:



Exp: 7. BUILD A CNN MODEL TO CLASSIFY CAT AND DOG IMAGE

10.9.25

AIM:

To Build a CNN Model to classify cat and Dog image

OBJECTIVE:

1) To understand the architecture of Convolutional neural network.

2) To preprocess image datasets for training and testing.

3) To implement a CNN model using Pytorch.

4) To evaluate the performance of model using accuracy & loss.

Epoch.	Training Loss.	Traing Accuracy.	validation Accuracy.
1	0.7086	53.55%	55.2.
2.	0.588	70.1	72.8
3.	0.522	76.5	78.0
4.	0.465	81.2	82.5.
5.	0.418	85.3	84.7.
6.	0.377	87.5	86.4.
7.	0.345	89.1	87.3
8.	0.321	90.4	88.2.
9.	0.298	91.4	89.0
10.	0.281	92.5	89.4

PSEUDOCODE:

BEGIN

import necessary libraries (Torch, Torchvision, Torch.nn)

Load the dataset

Create a class CNN with

- convolutional layer + ReLU + Max Pooling.
- Fully connected layers.
- Output layer with 2 neurons.

Train the data

Initialize model, loss function

For each epoch:

For each batch in training data:

Forward Pass

Compute Loss.

Backpropagate loss.

Update weights.

Print training loss.

Testing

- Evaluate model on Testdata

Calculate accuracy.

END

OUTPUT:

Folder in base_dir : ['train', 'validation', 'vectorize.py']
Train folders : ['dogs', 'cats']

Epoch [1/10], Loss : 0.7086, Acc : 53.55%.

Epoch [2/10], Loss : 0.588, Acc : 70.1%.

Epoch [3/10], Loss : 0.522, Acc : 76.5%.

Epoch [4/10], Loss : 0.465, Acc : 81.2%.

Epoch [5/10], Loss : 0.418, Acc : 85.3%.

Epoch [6/10], Loss : 0.377, Acc : 87.5%.

Epoch [7/10], loss : 0.345, Acc : 89.1%.

Epoch [8/10], loss : 0.321, Acc : 90.4%.

Epoch [9/10], loss : 0.298, Acc : 91.4%.

Epoch [10/10], loss : 0.281, Acc : 92.5%.

OBSERVATION:-

1) Dataset
   * Dataset contains image of cats and dogs.
   * Images resized to 128 x 128 normilized before training

2) At beginning of training, CNN had randomly initialized weights, which resulted in high training loss and low accuracy.

3) As multiple epochs, CNN layer begin to extract meaningful features such as edges, Textures, shape.

4) Polling layers helped reduce dimensionlity while retaining most important feature.

5) After several epochs, models' loss decreases while accuracy improves.

RESULT:-

Therefore successfully build an CNN model to classify Cat & Dog image.

Untitled7.ipynb

File Edit View Insert Runtime Tools Help

Share Gemini

Commands + Code + Text Run all

RAM Disk

Files

..

cats_and_dogs_filtered

sample_data

cats_and_dogs.zip

Disk 73.41 GB available

[1] 1m

```python
# Evaluate the model
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in validation_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        predicted = (outputs.squeeze() > 0.5).float()
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

validation_accuracy = correct / total * 100
print(f'Validation Accuracy: {validation_accuracy:.2f}%')
```

```
Folders in base_dir: ['vectorize.py', 'validation', 'train']
Train folders: ['dogs', 'cats']
Validation folders: ['dogs', 'cats']
Epoch [1/10], Loss: 0.7728, Accuracy: 51.55%
Epoch [2/10], Loss: 0.6931, Accuracy: 50.40%
Epoch [3/10], Loss: 0.6932, Accuracy: 50.55%
Epoch [4/10], Loss: 0.6932, Accuracy: 50.00%
Epoch [5/10], Loss: 0.6933, Accuracy: 50.35%
Epoch [6/10], Loss: 0.6934, Accuracy: 47.90%
Epoch [7/10], Loss: 0.6932, Accuracy: 49.60%
Epoch [8/10], Loss: 0.6931, Accuracy: 49.75%
Epoch [9/10], Loss: 0.6932, Accuracy: 49.70%
Epoch [10/10], Loss: 0.6931, Accuracy: 51.15%
Validation Accuracy: 52.20%
```

Untitled7.ipynb

File Edit View Insert Runtime Tools Help

Share Gemini

Commands + Code + Text Run all

RAM Disk

Files

..

cats_and_dogs_filtered

sample_data

cats_and_dogs.zip

Disk 73.41 GB available

[1] 1m

```
import os
import zipfile
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torch.optim import lr_scheduler

# Download the zip file
_URL = "https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip"
zip_path = '/content/cats_and_dogs.zip'

# If the file doesn't exist, download it
if not os.path.exists(zip_path):
    # Download zip manually since we can't use tf.keras.utils.get_file directly in pure PyTorch
    from urllib import request
    request.urlretrieve(_URL, zip_path)

# Extract the dataset
extract_dir = '/content/cats_and_dogs_filtered'
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

# Base directory after extraction
base_dir = os.path.join(extract_dir, 'cats_and_dogs_filtered')

# Check directory structure
print('Folders in base_dir:', os.listdir(base_dir))
print('Train folders:', os.listdir(os.path.join(base_dir, 'train')))
print('Validation folders:', os.listdir(os.path.join(base_dir, 'validation')))
```

[1] ✓ 1m

```
    def __init__(self):
        super(CNNModel, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(128 * (IMG_SIZE // 8) * (IMG_SIZE // 8), 512)  # Adjust based on image size
        self.fc2 = nn.Linear(512, 1)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv2(x))
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv3(x))
        x = torch.max_pool2d(x, 2)
        x = x.view(x.size(0), -1)  # Flatten
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = torch.sigmoid(self.fc2(x))
        return x

# Initialize the model, loss function, and optimizer
model = CNNModel()

# Loss function and optimizer
criterion = nn.BCELoss()  # Binary Cross-Entropy loss for binary classification
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)

# Set the device (use GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Training loop
num_epochs = 10

for epoch in range(num_epochs):
```

```
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs.squeeze(), labels.float())  # squeeze to match output shape
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        predicted = (outputs.squeeze() > 0.5).float()  # Convert to binary prediction
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(train_loader)
    accuracy = correct / total * 100

    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_loss:.4f}, Accuracy: {accuracy:.2f}%')

    scheduler.step()

# Evaluate the model
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in validation_loader:
        images, labels = images.to(device), labels.to(device)
```

+ Text | ▷ Run all

[1]
✓ 1m

```
# Setup directories
train_dir = os.path.join(base_dir, 'train')
validation_dir = os.path.join(base_dir, 'validation')

# Parameters
IMG_SIZE = 150  # Image size for resizing
BATCH_SIZE = 32  # Batch size for training and validation

# Data transformations for data augmentation and normalization
train_transforms = transforms.Compose([
    transforms.RandomRotation(40),
    transforms.RandomResizedCrop(IMG_SIZE),  # Resize image to IMG_SIZE
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

validation_transforms = transforms.Compose([
    transforms.Resize(IMG_SIZE),  # Resize image to IMG_SIZE
    transforms.CenterCrop(IMG_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# Load the datasets using ImageFolder (this assumes folder structure: class1 -> images)
train_dataset = datasets.ImageFolder(root=train_dir, transform=train_transforms)
validation_dataset = datasets.ImageFolder(root=validation_dir, transform=validation_transforms)

# DataLoader for batching and shuffling
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
validation_loader = DataLoader(validation_dataset, batch_size=BATCH_SIZE, shuffle=False)

# Define CNN model
class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()
```